1. MODELO DE CALIDAD DE USO

La calidad de Uso no existe en abstracto, definimos como parte del contexto el **usuario** (quién usará el software), **tareas** (qué tareas/funcionalidades puede usar el usuario final), **recursos tecnológicos** (sobre qué hardware, qué dispositivos, qué medios de comunicación va a correr nuestro producto o sistema ¿va a ser compatible con tablet, corre con 3G, 4G?, etc), **entorno físico social** (¿dónde va a correr nuestro producto? Una oficina, planta industrial, etc).

La calidad de producto es condición necesaria pero no suficiente para la calidad de uso.

Criterios de calidad de uso:

- **Efectividad: Requiere completitud y exactitud simultáneamente.**

El usuario puede completar una tarea sin errores en el resultado. Es decir, las actividades se pueden lograr y bien.

Efectividad = (Tareas completadas correctamente / Total de tareas intentadas) * 100

- **Eficiencia: ¿Cuánto le cuesta al usuario lograr su objetivo? Si la efectividad pregunta si se logró, la Eficiencia pregunta a qué precio el usuario logró hacer las tareas que debía hacer en el sistema.**

La norma habla de "recursos" en sentido amplio, no solo tiempo, sino todo lo que el usuario gasta para completar una terea: tiempo, esfuerzo cognitivo, acciones físicas, costo económico, esfuerzo emocional, soporte externo.

- **Satisfacción: Es la dimensión más subjetiva y emocional del modelo, mide cómo se siente el usuario, no solo qué logra o cuánto tarda.** Es también la más directamente ligada a la retención, el boca a boca y la adopción del sistema, es decir, que se masifique nuestro software.

Existen dos indicadores SUS (System Usability Scale) (El más usado) y NPS (NET PROMOTER SCORE) se hace una sola pregunta: ¿Qué tan probable es que recomiendes este sistema a un colega? (0-10) Sirve para recolectar información del usuario del servicio de que se está brindando.

- **Mitigación de riesgos: Mitigación de riesgo económico, de salud y seguridad y riesgo ambiental.**

Mitigación de riesgo económico: ¿El uso puede generar pérdidas financieras no deseadas?

Ejemplo: Una plataforma de inversión tiene un botón de "Comprar" y "Vender" muy cercanos y del mismo color, un usuario comete un error frecuente: vende cuando quería comprar.

Mitigación de riesgo de salud: ¿El uso puede dañar la salud o seguridad de personas?

Ejemplo: Una app de GPS que haga que el usuario se distraiga utilizando la app.

Mitigación de riesgo ambiental: ¿El uso puede generar daño al medio ambiente?

Ejemplo: Una app de gestión de flotas sugiere rutas que minimizan el tiempo de entrega, pero maximizan el consumo de combustible.

- **Cubrimiento de contexto: Las cuatro características anteriores de calidad de uso se evalúan en contexto de uso *definidos y conocidos.*** Pero en la práctica, los usuarios usan los sistemas en contextos no contemplados por el diseñador.

Ejemplo: Dispositivos no previstos, conexiones no previstas, usuarios no previstos, entornos no previstos.

Esta cualidad es la característica que cierra el modelo porque reconoce una verdad incómoda: los diseñadores nunca conocen perfectamente todos los contextos de uso reales. La pregunta es ¿qué pasa cuando los usuarios lo usan diferente?

2. PATRONES DE DISEÑO

2.1 Fundamentos

- **Patrón de diseño:** solución a un problema de diseño, **comprobada** en problemas similares y **reutilizable** en distintas circunstancias. Cada patrón describe un problema recurrente y el núcleo de su solución.
- **Para qué:** elementos reusables; no reinventar soluciones conocidas; vocabulario común entre diseñadores; estandarizar el diseño; facilitar el aprendizaje condensando conocimiento.
- **Qué NO buscan:** imponer alternativas de diseño, eliminar la creatividad, ni ser de uso obligatorio. Abusar o forzar patrones puede ser un error.
- **Categorías:** *de arquitectura* (esquema organizativo estructural fundamental), *de diseño* (esquemas para estructuras de diseño y sus relaciones), *dialectos* (bajo nivel, específicos de un lenguaje o entorno).
- **Clasificación (de los patrones de diseño):** *creación* (cómo se crean objetos), *estructurales* (composición de clases y objetos), *de comportamiento* (cómo interactúan y reparten responsabilidades).

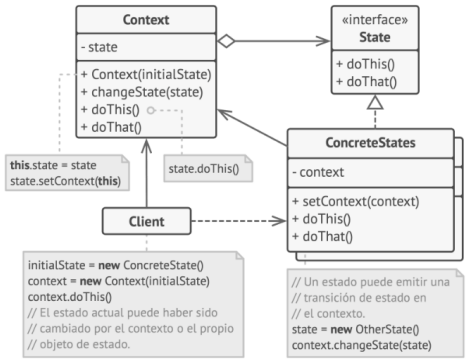
Catálogo (GoF):

Alcance	Creación	Estructura	Comportamiento
Clase	Factory Method	Adapter	Interpreter, Template Method
Objeto	Abstract Factory, Builder, Prototype, Singleton	Adapter, Bridge, Composite, Decorator, Facade, Proxy (y Flyweight)	Chain of Responsibility, Command, Iterator, Mediator, Memento, Observer, State, Strategy, Visitor

2.2 State (comportamiento)

- **Intención:** permitir que un objeto modifique su comportamiento cuando cambia su estado; parecerá que el objeto cambia de clase.
- **Motivación:** objetos cuyo comportamiento ante los mismos mensajes depende de su estado actual.
- **Aplicabilidad:** hay estados con operaciones comunes que se comportan distinto según el estado, y el código que varía en cada operación es mucho.
- **Participantes: Contexto** (Pedido; guarda una referencia al estado actual, delega en él y expone un setter), **interfaz Estado** (métodos que tengan sentido para todos los

estados) y **Estados concretos** (cada uno implementa su comportamiento y puede provocar la transición).



Problema del if/else (ejercicio Pedido): cada estado nuevo obliga a modificar **todos** los métodos, hay alto riesgo de errores por condicionales anidados, es difícil de testear (hay que recorrer los estados) y viola **Open/Closed**.

```
public interface EstadoPedido {
    void avanzarEstado(Pedido pedido);
    void cancelar(Pedido pedido);
    String obtenerEstado();
}

public class PedidoRecibido implements EstadoPedido {
    public void avanzarEstado(Pedido p) {
        p.setEstado(new PedidoEnPreparacion());
        // imprime: el pedido pasa a EN PREPARACION
    }
    public void cancelar(Pedido p) {
        p.setEstado(new PedidoCancelado());
        // imprime: pedido cancelado
    }
    public String obtenerEstado() { return RECIBIDO; }
}

public class Pedido {            // Contexto
    private EstadoPedido estado = new PedidoRecibido();   // estado inicial
    public void setEstado(EstadoPedido e) { this.estado = e; }
    public void avanzarEstado()  { estado.avanzarEstado(this); }
    public void cancelar()       { estado.cancelar(this); }
    public String obtenerEstado() { return estado.obtenerEstado(); }
}
```

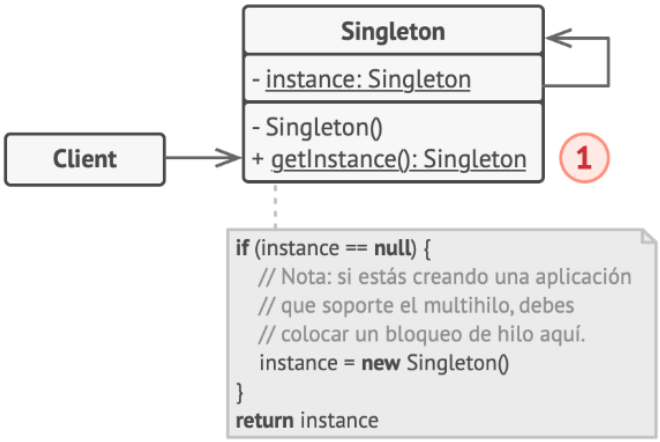
Estado	avanzarEstado()	cancelar()
Recibido	→ En preparación	→ Cancelado
En preparación	→ Listo	→ Cancelado
Listo	→ Entregado	No permitido (ya fue preparado)
Entregado	No permitido	No permitido
Cancelado	No permitido	No permitido

Tips para el Parcial (State)

- Cumple Open/Closed **en lo esencial**: agregar un estado exige tocar el estado vecino (su transición), pero no los demás ni el contexto.
- Cada operación debe **imprimir un mensaje descriptivo** (lo pide el enunciado).
- Relación con diagramas: un diagrama de estados es el modelo; State es su implementación.

2.3 Singleton (creación de objetos)

- **Intención:** asegurar que una clase tenga **una única instancia** y dar un **punto de acceso global**.
- **Motivación:** recursos físicos únicos, o datos que deben estar disponibles para toda la aplicación (ej.: configuración).
- **Aplicabilidad:** debe existir una sola instancia y el acceso a ella debe ser un punto bien conocido.
- **Estructura:** constructor **privado**; atributo estático con la instancia; método estático getInstance() que la crea la primera vez y luego devuelve siempre la misma. Si un método es de clase (static), los atributos que usa también deben ser de clase.




```
public class Configurador {
    private String url, baseDatos;
    private static Configurador miconfigurador;
    private Configurador(String url, String bd) { this.url = url; this.baseDatos = bd; }
    public synchronized static Configurador getInstance() {
        if (miconfigurador == null) {
            // lee url y baseDatos de un archivo de configuración
            miconfigurador = new Configurador(url, baseDatos);
        }
        return miconfigurador;
    }
}
```

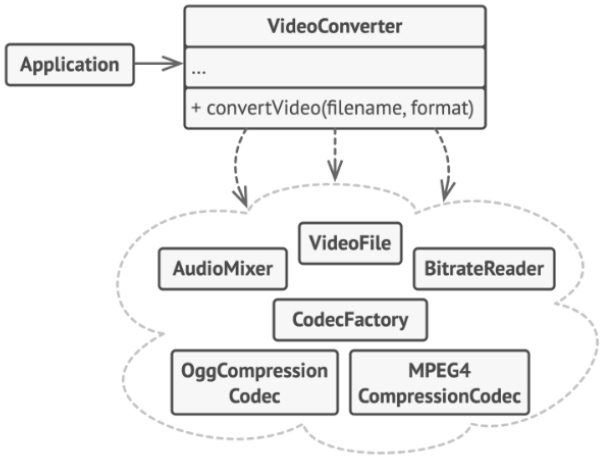
Tips para el Parcial (Singleton)

- Tres piezas que te piden dibujar: constructor privado, atributo estático, getInstance() estático.
- Si te preguntan «qué falla si se puede clonar»: aparecería una segunda instancia.

2.4 Facade / Fachada (estructural)

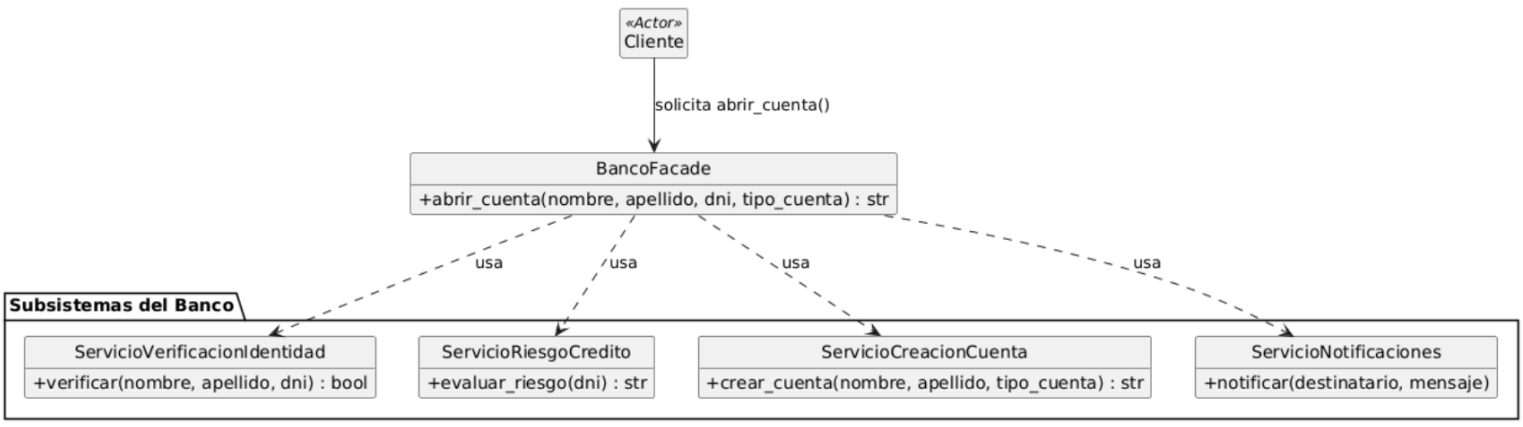
- **Intención:** interfaz **unificada** para un conjunto de interfaces de un subsistema.
- **Motivación:** estructurar en subsistemas reduce complejidad; queremos minimizar comunicación y dependencias entre subsistemas; una fachada ofrece una interfaz simplificada.
- **Participantes:** **Fachada** (sabe qué clase del subsistema atiende cada pedido y delega) y **Clases del subsistema** (implementan la funcionalidad y **no conocen** a la fachada).
- **Aplicabilidad:** interfaz única para un subsistema complejo; desacoplar el subsistema de sus clientes; dividir en capas.
- **Colaboraciones:** el cliente habla con la fachada, que reenvía a los objetos adecuados.
- **Consecuencias:** oculta los componentes del subsistema; **acoplamiento débil**; **no impide** que un cliente avanzado use las clases del subsistema directamente.
- **Contra:** puede volverse un objeto «todopoderoso» acoplado a todo.
- **Principio que promueve:** **Ley de Demeter** (mínimo conocimiento) y bajo acoplamiento.

En este ejemplo, el patrón **Facade** simplifica la interacción con un framework complejo de conversión de vídeo.



Un ejemplo de aislamiento de múltiples dependencias dentro de una única clase fachada.

Ejercicio (apertura de cuenta bancaria): la fachada recibe la solicitud y orquesta: (1) verificar identidad → si falla, excepción/mensaje; (2) evaluar riesgo → si es ALTO, rechazar; (3) crear cuenta; (4) notificar; (5) retornar el CBU. El cliente (Main) solo conoce la fachada.



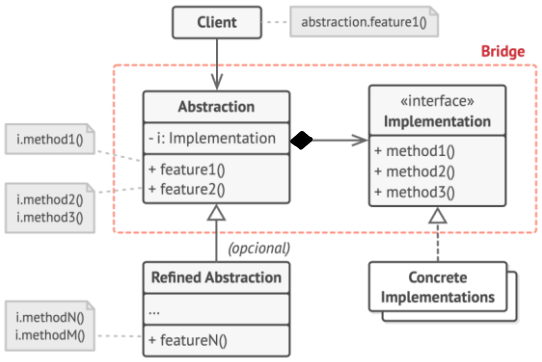
Preguntas de la diapositiva: antes el cliente conocía los 4 servicios; después solo la fachada. Si se agrega ServicioAntiFraude, el cambio se hace **solo en la fachada**. La fachada **no elimina** los subsistemas.

Tips para el Parcial (Facade)

- Facade **simplifica** el acceso; no agrega funcionalidad nueva ni oculta de forma obligatoria.
- Las clases del subsistema no tienen referencia a la fachada.

2.5 Bridge / Puente (estructural)

- **Intención:** desacoplar una abstracción de su implementación para que ambas varíen independientemente.
- **Motivación:** clases con **varias dimensiones de variación** (plataformas, formatos, dispositivos); con herencia se vuelve inmanejable. Bridge usa **composición en lugar de herencia**.
- **Aplicabilidad:** evitar un vínculo permanente entre abstracción e implementación; ambas pueden tener múltiples variaciones; cambios en una no deben afectar a la otra.
- **Participantes: Abstraction** (interfaz de alto nivel con una referencia a un Implementor), **RefinedAbstraction** (la extiende), **Implementation** (interfaz de las implementaciones), **ConcreteImplementation**.



Ejercicio (reportes): tipos de reporte (Ventas, Empleados, Inventario) × formatos (PDF, Excel, HTML). Con herencia: 3 × 3 = 9 clases (ReporteVentasPDF, ...). Con Bridge: **3 + 3**. La abstracción es *qué reporte* es; la implementación es *en qué formato* se genera; el cliente combina ambos.

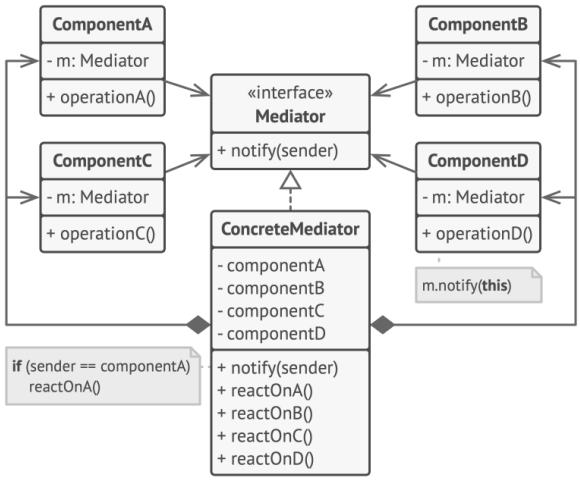
Promueve **Open/Closed**: agregar un formato o un reporte no toca lo existente.

Tips para el Parcial (Bridge)

- Palabra clave: **dos jerarquías que evolucionan por separado**, unidas por composición.
- Cuenta las clases: la explosión combinatoria con herencia vs. la suma con Bridge es el argumento típico.

2.6 Mediator / Mediador (comportamiento)

- **Intención:** reducir el acoplamiento directo entre objetos: se comunican a través de un **mediador central**.
- **Motivación:** muchas clases interconectadas hacen el sistema frágil y difícil de mantener.
- **Aplicabilidad:** muchos objetos interactúan de forma compleja pero bien definida; se quiere centralizar el control de la comunicación; clases difíciles de cambiar o reutilizar por estar muy acopladas.
- **Participantes:** **Mediator** (interfaz de comunicación, típicamente un método de notificación), **ConcreteMediator** (coordina y conoce a los colegas), **Colleague** (cada uno conoce solo al mediador) y **ConcreteColleague**.



Ejercicios: *Tráfico aéreo:* la torre de control es el mediador; aviones, pistas y servicios en tierra son colegas y se comunican solo con la torre. *Sala de chat:* Sala es el mediador; Usuario (colega) tiene enviarMensaje() y recibirMensaje(); la sala reenvía a los demás usuarios.

Tips para el Parcial (Mediator vs. Facade)

- Facade: comunicación **unidireccional** (cliente → subsistema) y el subsistema no conoce la fachada. Mediator: **bidireccional**, los colegas **sí** conocen al mediador.
- Riesgo del mediador (complemento general): puede concentrar demasiada lógica.

3. DIAGRAMAS DE CLASES, DAO y DTO

Tesis de la clase: cada clase debe tener **una sola razón para cambiar** (SRP). Si una entidad Libro buscara por ISBN, guardara, eliminara, actualizara stock, escribiera logs y se mostrara en la web y en un desktop, cambiaría por muchos motivos (modelo de BD, diseño de pantalla, logging...). Hay que separar responsabilidades.

DTO (Data Transfer Object)

- Es una **valija**: solo transporta datos. No piensa, no decide, no tiene lógica de negocio ni conexión a BD. Solo atributos, getters y setters.

- **Por qué si ya existe la entidad:** si la tabla tiene `precio_costo` (dato interno) y devolvés la entidad completa a la UI, ese dato viaja al frontend y cualquiera lo ve con el inspector del navegador. El DTO es el **filtro**: lleva solo lo que querés exponer.
- **Regla de oro:** si un DTO calcula, valida o llama a otra clase, **dejó de ser un DTO**.

DAO (Data Access Object)

- Es el **empleado del depósito**: el único autorizado a hablar con la base de datos. Todo pasa por él.
- Encapsula la lógica para llevar datos desde las clases del dominio (capa de negocio) hacia la persistencia (capa de datos) y viceversa. Operaciones típicas: añadir, actualizar, buscar, borrar (CRUD).
- **Objetivo:** la variedad de motores de BD no debe afectar a la lógica del software; bajar el acoplamiento y el impacto de upgrades o cambios.
- Se define como **interfaz**, no como clase concreta:

```
public interface LibroDAO {
    Libro buscarPorIsbn(String isbn);
    List<Libro> listarTodos();
    void guardar(Libro libro);
    void actualizarStock(String isbn, int nuevoStock);
    void eliminar(String isbn);
}
// Implementaciones: LibroDAOMySQL, LibroDAOMongo, LibroDAOFake (memoria, para tests)
```

Esto es **Inversión de Dependencias**: la lógica de negocio depende de una abstracción, no de una implementación. Consecuencias: se cambia de BD sin tocar el resto, se testea la lógica sin BD real y se puede desarrollar en paralelo mientras infraestructura decide el motor.

Error típico: el DAO devuelve un DTO

`LibroDTO buscarPorIsbn(...)` dentro del DAO mezcla las dos responsabilidades. El mismo libro se muestra distinto en tres pantallas (con precio, sin precio, solo título). Si el DAO devuelve un DTO fijo, quedás atado. **El DAO devuelve la entidad; el DTO se arma en cada caso.**

Pasos para implementar el patrón DAO:

1. Identificar las clases de dominio (business objects).
2. Una clase DTO por clase de dominio a utilizar, solo con los atributos necesarios y sin lógica de negocio.
3. Una interfaz DAO por clase de dominio, con CRUD y los métodos extra necesarios.
4. Implementar la clase concreta de cada interfaz.

Ejercicio Candidato (propuesta de solución)

CandidatoDAO (CU006 importar, CU025 evaluar, CU026 completar cuestionario): `guardar(Candidato)` (importación), `buscarPorNroCandidato(int)`, `buscarPorDocumento(tipoDoc, nroDoc)` (evitar duplicados al importar), `listarTodos()` (consultor), `actualizar(Candidato)` (persistir el token generado), `buscarPorToken(String)` (ingreso al cuestionario).

CandidatoGestionDTO (consultor, CU002/CU025): nroCandidato, apellido, nombre, tipoDocumento, nroDocumento, email, nacionalidad.

CandidatoCuestionarioDTO (el propio candidato, CU026): solo lo mínimo, por ejemplo nroCandidato, nombre y apellido. Sin email, documento, fecha de nacimiento ni token.

Tips para el Parcial

- DAO = acceso a datos (capa de datos); DTO = transporte entre lógica y presentación. En el TP: DAOs en la capa de datos, gestores y clases en la lógica, DTOs hacia el frontend.
- Dos preguntas clásicas: ¿por qué el DAO es una interfaz? (múltiples implementaciones + DIP) y ¿por qué el DAO no devuelve DTO? (queda atado a una vista).

4. PRINCIPIOS SOLID

Son **transversales a todo el diseño**. Buscan código limpio (fácil de entender), mantenible (correcciones simples) y extensible; reducen el acoplamiento y anticipan el cambio en lugar de temerle.

Letra	Principio	En una frase
S	Single Responsibility	Una clase, una razón para cambiar
O	Open/Closed	Extender sin modificar lo existente
L	Liskov Substitution	Una subclase no debe romper el contrato del padre
I	Interface Segregation	Interfaces chicas y específicas
D	Dependency Inversion	Depender de abstracciones, no de detalles

S — Single Responsibility (SRP)

Una clase/módulo tiene una única responsabilidad (**alta cohesión**). Ejemplo de la diapositiva: ReportManager que genera el informe, lo guarda en archivo **y** envía mail tiene tres razones para cambiar (formato, destino, proveedor de email). Se separa en ReportGenerator, ReportSaver, EmailService, y ReportManager solo **orquesta**.

O — Open/Closed (OCP)

Entidades abiertas para la **extensión**, cerradas para la **modificación**. Si no toco lo que ya anda, me ahorro modificar y retestear. Ejemplo: CalculadoraDeArea con if (figura is Circulo) ... else if (figura is Rectangulo) viola OCP porque cada figura nueva obliga a modificar el método. Solución: interfaz IFigura con CalcularArea(); cada figura sabe calcular su área y una figura nueva es una clase nueva.

Ventajas: evita romper funcionalidad probada, reduce el impacto de cambios, favorece abstracciones e interfaces, fomenta reutilización y mantenibilidad.

L — Liskov Substitution (LSP)

Una subclase, en cualquier contexto, debe poder **sustituir** a la clase padre sin romper el contrato.

- Violación: Ave tiene Comer() y Volar(). Pinguino: Ave hereda Volar(), pero lo implementa como throw NotImplementedException. Ave a = new Pinguino(); a.Volar(); **rompe**. «Es un» en el modelo, pero no cumple el contrato.
- Solución: Ave solo declara lo que **todas** las aves cumplen (Comer()); Volar() va en una interfaz IVolador que implementan Aguila (y no Pinguino).

Ventajas: evita errores de herencia, favorece el polimorfismo, mejora la reutilización y fomenta un contrato entre subclases y padre.

I — Interface Segregation (ISP)

Una clase no debe verse obligada a depender de métodos que no usa. **Mejor muchas interfaces chicas y específicas que una grande y genérica.** Violación: IVehiculo con Conducir(), Volar(), Navegar() obliga a Auto a tener métodos vacíos o con excepción. Solución: IConducible, IVolador, INavegable; cada clase implementa solo las que necesita (una clase puede implementar varias).

Ventajas: alta cohesión y menor acoplamiento, más flexibilidad y mantenibilidad, evolución más fácil.

D — Dependency Inversion (DIP)

- Los módulos de **alto nivel** (orquestan la lógica de negocio: controlador, gestor) no deben depender de los de **bajo nivel** (ejecutan detalles: acceso a BD, notificaciones). **Ambos dependen de abstracciones.**
- Las abstracciones no dependen de los detalles; **los detalles dependen de las abstracciones**.
- **Abstracción** = interfaz o clase abstracta (un contrato sin implementación).
- **En el TP:** Gestores = alto nivel; **IDAO (interfaz) = abstracción**; DAOs = bajo nivel. Gestores y DAOs dependen de la interfaz.

Cómo se implementa:

- **IoC (Inversión de Control):** el flujo y la creación/gestión de objetos se delega a un contenedor, framework o infraestructura externa.
- **DI (Inyección de Dependencias):** la implementación se inyecta desde afuera en lugar de crearla adentro. Formas: por **constructor** (la más común), por propiedad, por método.

```
interface NotificationService { void send(String message); }
class EmailService implements NotificationService { /* envía el email */ }
class OrderService {
    private NotificationService notifier;              // depende de la ABSTRACCIÓN
    OrderService(NotificationService n) { this.notifier = n; }   // DI por constructor
}
```

Tips para el Parcial (SOLID)

- **DIP ≠ IoC ≠ DI:** DIP es el principio; IoC y DI son las formas de lograrlo.
- En ejercicios de «identificá la violación»: muchos if/else por tipo = OCP; método que lanza excepción en la subclase = LSP; métodos vacíos por interfaz gorda = ISP; clase que hace new de una clase concreta de bajo nivel = DIP; clase con varios motivos de cambio = SRP.

- ISP y SRP están emparentados (cohesión), pero ISP habla de **interfaces** y SRP de **clases**.

5. VISTA DINÁMICA: MÁQUINA DE ESTADOS

- **Vistas:** interacciones (relevante para el nivel de acoplamiento que puedo llegar a tener en el sistema), actividades y máquinas de estados.
- **Diagrama de estados:** modela la vista de **un único objeto**, tratado como instancia aislada que detecta eventos y responde. Se puede implementar con el patrón **State**.
- **Tres elementos clave:**
  - **estado** (condición durante la vida del objeto: satisface una condición, realiza una actividad o espera un evento),
  - **evento** (ocurrencia con ubicación en tiempo y espacio) y
  - **transición** (paso de un estado a otro; es instantánea, el sistema siempre está en un estado conocido).
- **Estado inicial:** representa la creación del objeto; hay **uno solo**. **Estado final:** desaparición del objeto; puede haber 0..N.
- **Transición lambda:** no requiere un evento para habilitarse.
- **Condición de control (guard):** expresión lógica verdadera/falsa que se evalúa cuando ocurre el evento. Notación: evento [condición] / acción.
- **Acciones en transiciones:** se ejecutan al provocarse la transición, pueden llamar operaciones del objeto o de otros visibles y son **atómicas** (no las interrumpe un evento).
- **Acciones en estados:** al entrar (entry), al salir (exit), mientras está en el estado (do) y al ocurrir un evento estando en él.
- **Estados compuestos:** un superestado con subestados y su propio estado inicial (ej.: cajero automático, estado «activo»).
- **Historia (H):** recuerda el último subestado visitado del superestado y lo recupera al volver a entrar.

Tips para el Parcial

- Un solo estado inicial, varios finales posibles.
- No confundir **evento** (algo que ocurre) con **acción** (algo que el objeto hace en respuesta).

6. COMPONENTES, PAQUETES, SUBSISTEMAS Y DESPLIEGUE

6.1 Componentes

- **Componente:** parte ***modular*****, desplegable y reemplazable** de un sistema que **encapsula** su implementación y expone un conjunto de ***interfaces***. Es ***cohesivo***: un bloque significativo de comportamiento o estructura de un sistema mayor.
- **Interfaz proporcionada** (provista): lo que el componente ofrece. **Interfaz requerida:** lo que necesita de otros. En el modelado hay que identificar cuáles provee y cuáles requiere; las interfaces deben cumplir **ISP**.
- **Notación:** estereotipo «component», con puertos y conectores.
- **Ejemplo (diapositiva, software de correo):** Email Inbox, Email Management y Email Outbox, con interfaces como *receive email*, *collect email*, *send email*, un puerto de

servicio y un puerto de gestión (*monitor operation*). Otro ejemplo: un componente de acceso a datos que **provee** una interfaz CRUD y **requiere** una conexión a la BD.

6.2 Paquetes de diseño y capas

- **Paquete:** agrupa elementos relacionados para estructurar el modelo en partes más manejables. Se usa para reflejar la **arquitectura**: capas lógicas (**layers**), capas físicas (**tiers**) o **subsistemas**. (En una BD, cada esquema puede modelarse como un paquete.)
- **Visibilidad: pública** = cualquier elemento puede asociarse o usarse desde cualquier clase, dentro o fuera del paquete. **Privada** = solo visible para las clases del mismo paquete. Se relaciona con **encapsulamiento / ocultamiento de información**.

Concepto	Qué es	Ejemplo
Capa lógica (layer)	Agrupación **conceptual** de elementos que separa responsabilidades	Presentación, Lógica de negocio, Acceso a datos (o Vista / Controlador / Modelo)
Capa física (tier)	**Dónde se despliega**: nodos o procesadores	Servidor de aplicaciones, servidor de BD

- **Dependencia en cascada:** Presentación → Lógica de negocio → Acceso a datos. **No al revés**, porque generaría dependencia y acoplamiento.
- **Objetivo del aislamiento:** ante el cambio de un componente dentro de un paquete, que los errores **no se propaguen** a otras capas.
- **En el TP:** DAOs en acceso a datos; gestores y clases de dominio en lógica de negocio; DTOs entre lógica y presentación; frontend en presentación.

6.3 Subsistema

- Es un **paquete** que, además de contener clases (y posiblemente otros paquetes), **exhibe comportamiento accesible a través de una o más interfaces**. Estereotipo «subsystem» (ej.: *Firma digital*).
- El comportamiento lo implementan clases y paquetes **internos**, que **no son accesibles desde afuera**. Muestra el contrato, no el detalle.
- **Paquete vs. subsistema:** el paquete solo agrupa; el subsistema agrupa **y** expone comportamiento tras interfaces (es lo que luego una **Facade** puede representar).

6.4 Cohesión y acoplamiento

Tipo	Descripción
Cohesión funcional	Todos los elementos contribuyen a una única función bien definida (la mejor)
Cohesión de capa	Se agrupan elementos que pertenecen a la misma capa
Cohesión de comunicación	Componentes que se comunican o trabajan sobre los mismos datos para un propósito específico

Tipo	Descripción
Acoplamiento de contenido	Un componente accede o depende de la **estructura interna** de otro (el peor)
Acoplamiento de control	Un componente pasa un **flag** que cambia la lógica del otro. Si pueden cambiar independientemente, hay que separarlos
Acoplamiento externo	Dependen de un formato, protocolo o dispositivo impuesto desde afuera

El envío de mensajes con datos es un acoplamiento **de datos/mensajes**, el más débil y deseable.

6.5 Principios para organizar componentes

- **PER** (equivalencia de reutilización/liberación): la unidad de reutilización es la unidad de **liberación**. Lo que se reutiliza debe versionarse y liberarse como conjunto, sin romper contratos con el tiempo.
- **PCC** (cierre común): lo que **cambia junto** y por las mismas razones va en el **mismo** paquete (ej.: el acceso a la capa de datos).
- **PRC** (reutilización común): lo que se **reutiliza junto** va junto; lo que no, se **separa** (ej.: *guardar habitación* y *gestionar usuario* no se reutilizan juntos, así que van en paquetes distintos).

6.6 Diagrama de despliegue

- Muestra la distribución del sistema en **nodos de procesamiento**, los **enlaces** entre ellos y los **procesos** que corren en cada uno. Modela la vista de despliegue **estática**.
- **Nodo:** recurso computacional con capacidad de memorizar y procesar («device», «executionEnvironment» como Tomcat, Docker, un motor de BD).
- **Artefacto:** pieza **física** de información usada o producida (archivo): .war, .jar, .exe, scripts, web.xml (deployment spec).
- **Relaciones:** «deploy» (el artefacto se despliega en un nodo), «manifest» (el artefacto manifiesta/implementa un componente) y **vínculos de comunicación** etiquetados con el protocolo (TCP/IP, HTTP, HTTPS).
- **Ejemplos de la diapositiva:** Application Server con Tomcat 5.5 (book_club_app.war, user_services.jar) conectado por TCP/IP a un Database Server con Oracle 10g y varios schemas; y un Docker Host con contenedores (web, api, base MariaDB) comunicados por HTTP/GraphQL.

6.7 Ejercicio CampusCoffee (propuesta)

- **Componentes:** App Móvil, Gestión de Pedidos, Menú, **Pagos**, Cocina (pantalla), Notificaciones, Persistencia. Pagos **provee** una interfaz propia IPago y **requiere** la interfaz de la pasarela externa (MercadoPago), que queda oculta. Gestión de Pedidos **requiere** IPago, IMenu, INotificacion, IPedidoRepo.
- **Paquetes por capa:** Presentación (app móvil, pantalla de cocina) → Lógica de negocio (pedidos, menú, pagos, notificaciones) → Acceso a datos (DAOs).

- **Subsistema: Pagos**, porque expone un comportamiento (procesar pago) tras una interfaz y oculta la integración externa (se puede cambiar de pasarela sin afectar al resto).
- **Despliegue:** Dispositivo móvil (artefacto de la app) → HTTPS → Nodo Servidor de aplicaciones con «executionEnvironment» Tomcat 11 (campuscoffee.war con «manifest» de los componentes y «deploy» al nodo) → TCP/IP (JDBC) → Nodo Servidor de BD con PostgreSQL 18; y desde el servidor de aplicaciones → **HTTPS** → Pasarela de pago externa.

Tips para el Parcial

- Si te piden «capas» pensar en **lógico**; si te piden «nodos / dónde corre» pensar en **físico (tiers)**.
- «manifest» conecta artefacto y componente; «deploy» conecta artefacto y nodo. No los inviertas.
- Subsistema = paquete **con interfaces y comportamiento**; no es solo una carpeta.
- Cohesión: cuanto más, mejor. Acoplamiento: cuanto menos, mejor.

7. ARQUITECTURA DE SOFTWARE

7.1 Conceptos

- **Arquitectura:** planificación basada en modelos, patrones y abstracciones, previa a cualquier implementación. En su forma simple: la estructura de organización de los **componentes**, cómo **interactúan** y las **estructuras de datos** que usan. Sirve de base para el diseño detallado. «Los **datos** son el activo más importante» y el cliente debe entender cómo los componentes resuelven los requerimientos.
- **Para qué sirve su representación:** analizar si el diseño cumple los requerimientos, considerar alternativas cuando cambiar es fácil y reducir riesgos de construcción. **No** se especifican las propiedades internas de los componentes.
- **Vistas (AD):** conjunto de productos que reflejan distintos puntos de vista; cada perspectiva atiende la preocupación de un participante. El **género arquitectónico** (tipo de problema: salud, streaming, facturación, IA...) orienta la estructura.
- **Estilo arquitectónico**: describe una categoría de sistemas que incluye (1) **componentes** que cumplen una función, (2) **conectores** que permiten comunicación, coordinación y cooperación, (3) **restricciones** que definen cómo se integran y (4) **modelos semánticos** que permiten entender las propiedades generales del sistema.

7.2 Las 4 dimensiones (Richards y Ford)

1. **Características de la arquitectura** (requerimientos no funcionales): disponibilidad (acceso cuando se necesita), fiabilidad, testeabilidad, escalabilidad, seguridad, agilidad (responder rápido al cambio), tolerancia a fallos (recuperarse sin dejar estados inconsistentes), elasticidad, recuperabilidad, performance, despliegue (desplegar por componentes) y facilidad de aprendizaje. **Escalabilidad** = soportar crecimiento sostenido de usuarios; **elasticidad** = absorber picos de carga. No se pueden contemplar todas: se **priorizan** según el género.
2. **Estructura:** el estilo usado para organizar componentes (capas, microkernel, microservicios...).

3. **Decisiones arquitectónicas: reglas** que deben cumplirse en todos los componentes. Ej.: en capas, la presentación **no** puede hablar con la persistencia.
4. **Principios de diseño: recomendaciones** o buenas prácticas, no obligatorias. Ej.: preferir mensajería **asíncrona** entre servicios para mejorar performance; asumir que las llamadas remotas pueden fallar (timeouts, reintentos).

Capas abiertas y cerradas (decisión): en una capa **cerrada**, una petición debe pasar por ella para llegar a la siguiente (presentación y negocio son cerradas). Una capa **abierta** (ej.: servicios) puede saltearse. Según la diapositiva, solo las capas de negocio y servicios acceden a persistencia.

7.3 Partición arquitectónica

- **Técnica (Technical Partitioning):** componentes por capa técnica (presentación, negocio, persistencia). + reutilización y comprensión por rol; − baja cohesión de negocio, dificulta pasar a distribuida.
- **De dominio (Domain Partitioning):** componentes por área de negocio (DDD – Domain-Driven-Design). + cohesión, bajo acoplamiento entre dominios, base para microservicios; − exige conocer el negocio y puede duplicar infraestructura técnica.

7.4 Monolítico vs. distribuido

- **Monolítico:** una sola unidad de despliegue; los componentes corren en el mismo proceso.
- **Distribuido:** servicios independientes que se comunican **por red** (API/REST, gRPC), cada uno con su código y, a veces, su BD.

8 falacias de la computación distribuida: (1) la red es confiable, (2) la latencia es cero, (3) el ancho de banda es infinito, (4) la red es segura, (5) la topología no cambia, (6) hay un solo administrador, (7) el costo de transporte es cero, (8) la red es homogénea.

7.5 Estilos arquitectónicos

Centrado en los datos (repositorio/pizarrón). Un almacén de datos central al que acceden clientes que agregan, modifican o consultan datos. ✅ Simple, consistente (ACID), curva de aprendizaje baja. ❌ Alto acoplamiento al esquema, difícil de testear, cuello de botella de escalabilidad, la lógica se dispersa.

En capas (el más usado, monolítico, partición técnica). Cada capa tiene un rol específico y un cambio en una capa no afecta a las otras **si las capas están cerradas**. Capas típicas: **Presentación** (UI y peticiones HTTP), **Negocio** (reglas y lógica), **Persistencia** (mapeo y acceso a datos), **Base de datos**. Variantes: 2, 3 o 4 capas (ej.: unir persistencia y BD). Componentes clásicos: capa de interfaz de usuario, de aplicación, de utilerías y de núcleo.

MVC (arquitectura/estilo para la presentación): **Controlador** (gestiona las solicitudes del usuario, selecciona el comportamiento del modelo y la vista a mostrar), **Modelo** (encapsula funcionalidad, objetos de contenido y estados de la aplicación) y **Vista** (prepara datos del modelo, solicita actualizaciones y presenta lo que selecciona el controlador).

Flujo de datos (tuberías y filtros). Los datos pasan por **filtros** (cada uno transforma) conectados por **tubos**; cada etapa solo recibe la salida de la anterior. Ej.: ETL (Extracción,

transformación y carga), edición de video, procesamiento de logs. + Reutilización y paralelización; − latencia y poca flexibilidad para interacciones bidireccionales.

Microkernel (plug-ins). Sistema central (funcionalidad mínima: ciclo de vida, configuración) + **plug-ins** independientes que lo extienden mediante interfaces/puntos de extensión. El core puede estar en capas (partición técnica) o modular (de dominio). Ej.: editor de código con extensiones. (VsCode)

Basado en servicios. Partición de **dominio**; pocos servicios de grano grueso, desplegados por separado, generalmente con **una BD compartida**. Variantes de UI: única, por dominio o por servicio. Puede tener una **capa de API (proxy o gateway)** que limita a qué APIs accede cada usuario. Buena gestión de despliegue y asignación de recursos.

Solicitud-respuesta. El cliente envía una solicitud y espera una respuesta; un **orquestador de solicitudes** la dirige a procesadores de solicitud con su estado. Es la base de la web.

(Event-Driven Architecture)

Basado en eventos (EDA). Un **evento** es un cambio de estado; puede llevar datos o solo identificadores. Componentes: evento iniciador, canal de eventos y procesadores de eventos. Acoplamiento débil entre productor y consumidor; permite agregar consumidores sin afectar a los existentes. **Broker:** sin coordinador central, cada componente reacciona de forma autónoma (ej.: *order-created* dispara pago, inventario, notificación). **Mediator:** hay un mediador central que coordina flujos complejos de varios pasos.

Microservicios. Aplicación como conjunto de servicios pequeños e independientes; cada uno con **su propio código, datos (BD) y dependencias**, detrás de una capa de API.

+ Escalabilidad independiente, resiliencia (tolerancia a fallos), despliegue independiente.
− Complejidad operativa y de red (las 8 falacias), consistencia de datos y costos de infraestructura.

Ej.: e-commerce, streaming, servicios financieros.

Stateless vs. stateful: *stateless* no guarda estado de sesión en el servidor (ej.: **JWT** firmado en el cliente; cualquier nodo valida la firma); *stateful* guarda la sesión en memoria del servidor (JSESSIONID) y requiere **sticky sessions** con balanceador. Stateless escala y tolera fallos mejor.

7.6 Escalabilidad

- **Vertical (hacia arriba):** agregar recursos físicos (CPU/RAM) al mismo servidor. Esfuerzo mínimo, pero **límite fijo del hardware** (y sigue siendo un punto único de falla).
- **Horizontal (hacia los lados):** agregar **nodos** formando un **clúster**; el trabajo se reparte. Más potente y más complejo; requiere un **servidor primario** que administre el clúster (y balanceador de carga).

7.7 Arquitectura en el Proceso Unificado

Modelo	Vista arquitectónica	Contenido
Casos de uso	**Vista de casos de uso**	Casos de uso **arquitectónicamente significativos** (críticos o urgentes), con diagrama y descripción
Análisis	**Vista lógica**	Paquetes de análisis y dependencias, clases clave (entidad, interfaz, control) y realizaciones de los CU de la vista
Diseño	**Vista lógica**	Capas lógicas/físicas y subsistemas con diagramas de clases; mecanismos generales (persistencia, distribución, rendimiento) con diagramas de clases e interacción
Diseño	**Vista de despliegue**	Nodos, dispositivos y vínculos, en un diagrama de despliegue

Diseñar la arquitectura: identificar nodos y configuraciones de red; descomposición en capas y subsistemas con sus interfaces; mecanismos generales; clases de diseño significativas; reutilización de partes de sistemas parecidos; uso de software de terceros (librerías, APIs, frameworks).

7.8 ¿Qué propondrías? / ¿Qué respondés? (respuestas razonadas)

1. **Panadería online** (3 devs, presupuesto chico, 3 meses): **monolito en capas** (simple y barato; microservicios sería sobrediseño).
2. **Editor de código con extensiones: Microkernel / plug-ins**.
3. **Logs que se limpian, filtran, pasan a JSON y se agregan: Flujo de datos (tuberías y filtros)**.
4. **Presentación consulta persistencia directamente (capas cerradas):** rompe la regla de capas cerradas; acopla la UI a la persistencia, duplica lógica y hace que un cambio en datos se propague a la UI.
5. **Black Friday con servidor del doble de CPU/RAM: escalamiento vertical**; limitación: **techo del hardware** (y punto único de falla).
6. **Pedidos llama a Pagos sin timeouts ni reintentos:** falacia «**la red es confiable**».

Tips para el Parcial

- Decisión = obligatoria; principio = recomendación.
- Basado en servicios ≠ microservicios ≠ SOA: servicios gruesos + BD compartida vs. servicios finos + BD propia.
- EDA broker vs. mediator: sin coordinador vs. con coordinador central.
- Escalar horizontal exige que la app sea stateless o comparta el estado.

8. PRUEBAS DE SOFTWARE: FUNDAMENTOS Y DISEÑO DE CASOS

8.1 ¿Qué significa probar?

- **Definición correcta:** probar es el proceso de **ejecutar un programa con la intención de encontrar errores**. Es una actividad **destructiva** (vs desarrollar que es constructivo).
- **Definiciones INCORRECTAS:** (1) probar es demostrar que no hay errores; (2) probar es mostrar que el programa realiza correctamente sus funciones; (3) probar es el proceso que lleva a confiar en que el programa hará lo que debe. La prueba es **una de las** cosas que ayudan a confiar, pero no es su propósito.
- **Un caso de prueba exitoso es el que descubre un error** (el resultado obtenido no coincide con el esperado). Cada error hallado ahora es un costo evitado después.
- **Dijkstra:** las pruebas pueden mostrar la presencia de errores, pero nunca su ausencia. Probar que no hay errores es una meta **imposible**; encontrar errores es una meta **alcanzable**.
- La **prueba exhaustiva es imposible**: hay que elegir el subconjunto de casos con mayor probabilidad de detectar errores.

8.2 Error, defecto y falla

- **Error (mistake):** equivocación de una persona (ej.: interpretar mal la especificación).
- **Defecto (bug/fault):** está en el código o el diseño (ej.: a + b puede desbordar int).
- **Falla (failure):** se observa al ejecutar (ej.: un isósceles válido es rechazado).
- **Caso de prueba** = entradas + condiciones de ejecución + **resultado esperado**. Sin resultado esperado no se puede decidir si hubo falla. Hay falla en **dos sentidos**: no hace lo que debe, o hace lo que no debe.

8.3 Psicología y economía de la prueba

- **¿Quién prueba?** Otra persona (otro programador que no escribió esa parte, o un tester). El autor tiene preconceptos, empatía con su obra y tiende a probar «solo en positivo». La independencia mejora la prueba; vale también para empresas. Solo es aceptable que prueben los mismos programadores en métodos ágiles. El tester debe ser metódico y estar motivado a encontrar errores.
- **Economía:** probar es una **inversión**, no hacerlo es un gasto. Como es imposible probar todo, la pregunta es **cuántos casos y para qué**. Probar y no encontrar nada **no prueba** que no haya errores: solo indica que esos casos no los hallaron.
- **Consecuencia comercial:** cuando el cliente percibe el error percibe también la falibilidad de quienes desarrollaron el sistema (pérdida de confianza y de ventas futuras).

8.4 Principios de la prueba (10)

1. El **resultado esperado es parte del caso** y se define **antes** de ejecutar (si no, el ojo ve lo que quiere ver).
2. Evitar que el **autor** pruebe su programa (sesgo y misma interpretación errónea de la especificación).
3. Una **empresa** no debería probar sus propios programas (alcance, costo, tiempo, calidad).

depende de qué se busca priorizar de estos items.

4. **Inspeccionar a fondo** cada resultado.
5. Escribir casos para entradas **inválidas e inesperadas**, no solo para las válidas.
6. Verificar también que **no hace lo que no debe** (es la otra mitad del problema).
7. **Evitar casos desechables:** conservarlos y automatizarlos (inversión, regresión).
8. **No asumir que no habrá errores:** planificar esperando encontrarlos.
9. **Los errores se agrupan:** más errores hallados en una sección, más probable que queden otros.
10. Probar es una tarea **creativa** y un desafío intelectual.

8.5 Inspecciones y revisiones (prueba humana, sin computadora)

- Lectura/inspección visual del programa por un **equipo**. El objetivo es **encontrar errores, no soluciones** (la corrección se decide después). Dan feedback al programador y aprendizaje al equipo; son muy útiles en el **mantenimiento**.
- Equipo típico de 4: moderador (programador experto, no autor), programador (autor), diseñador y especialista en pruebas; sesiones de 90 a 120 minutos, con lista ayuda-memoria y actitud sin ego. No son eficaces para errores de nivel superior.

8.6 Estrategias: caja negra y caja blanca

	Caja negra	**Caja blanca**
Casos derivados de	Solo la **especificación**	La **estructura interna** del código
Técnicas	Particiones de equivalencia, valores límite, conjetura de errores	Cobertura de sentencias, decisiones, condiciones
Cuándo	Pruebas de función; código complejo con muchos caminos	Pruebas de unidad; código simple; crítico (dinero, auditorías)

Ninguna es mejor en sí: se **combinan**. Diseñar casos al azar también es una metodología, pero muy pobre.

8.7 Caja negra

Un buen caso de prueba tiene probabilidad razonable de encontrar errores, **reduce** el número de otros casos necesarios y **cubre** un conjunto extenso de otros casos posibles.

Particiones de equivalencia. Si un caso de una clase detecta un error, se espera que cualquier otro de la misma clase lo detecte: **una clase, un representante**.

Procedimiento: (1) identificar condiciones externas de entrada; (2) partir cada condición en clases válidas e inválidas; (3) **numerar** cada clase; (4) casos **válidos: combinar** tantas clases válidas no cubiertas como se pueda; (5) casos **inválidos: uno por clase inválida**, con el resto de los datos válidos.

Condición de entrada	**Clases válidas**	**Clases inválidas**
Rango de valores	1	2 (por debajo y por encima)

Condición de entrada	Clases válidas	Clases inválidas
Cantidad específica de valores	1	2 (menos y más)
Conjunto de valores tratados distinto	Una por valor	1
Condición «debe ser»	1	1

Subdividir una clase si el programa trata distinto a algunos de sus elementos. **Ejemplo:** cantidad entre 1 y 999 y primer carácter letra: válido (500, Ana) cubre las clases 1 y 4; inválidos (0, Ana), (1000, Ana), (500, 7up).

Análisis de valores límite. Los casos en los bordes rinden más. Se prueba **justo en y justo fuera** de cada margen de cada clase, **de entrada y de salida**. Dominio 1 a 255: 0, 1, 255, 256. Rango: casos en los extremos y justo más allá. Conjuntos ordenados: primer y último elemento. También en salidas calculadas (ej.: contribución de $0 a $500.000: $0, $500.000, negativa, mayor).

Conjetura de errores. Intuición y experiencia sobre errores probables. «Sospechosos habituales»: lista vacía, una sola entrada, todos los valores iguales, ya ordenada, vacío/nulo/solo espacios, cero, negativos, valores máximos, caracteres especiales y acentos, fechas límite (29 de febrero), duplicados, orden inverso.

Estrategia combinada: clases de equivalencia (entrada y salida) → **valores límite** → **conjetura de errores**. No garantiza hallar todos los errores, pero es un compromiso razonable. Diferencia: equivalencia elige un representante; límite explora los **bordes** de las clases.

8.8 Caja blanca: cobertura lógica

La prueba perfecta recorrería todos los caminos, pero es inviable. Ejecutar cada sentencia al menos una vez es **necesario, pero no suficiente**.

Criterio	Exige	Observación
Sentencias	Cada línea se ejecuta al menos una vez	El más débil. Ej.: if (x > 0) {...} con solo x = 1 da 100% y nunca probó x ≤ 0
Decisiones (ramas)	Cada decisión toma V y F	Incluye sentencias. Ej.: x = 1 y x = -1
Condiciones	Cada condición individual toma V y F	No siempre cubre las decisiones
Condición/decisión	Ambos criterios a la vez	Incluye los dos anteriores

Ejemplo clave: if (a > 0 || b > 0) con (1, -1) y (-1, 1) cubre condiciones (cada una toma V y F), pero la decisión fue **siempre verdadera**: nunca se probó la rama falsa. En cambio (1, 1) y (-1, -1) alcanzan condición/decisión. Una cobertura del 100% no prueba que no falte funcionalidad: se combina con caja negra.

8.9 Caso de estudio: clasificar triángulos

El programa lee tres enteros por línea y clasifica: **equilátero** (a = b = c), **isósceles** (dos iguales, el desigual menor que la suma de los iguales) o **escaleno** (a ≠ b ≠ c). Existe un triángulo si la suma de los dos lados menores es **mayor** que el mayor.

Casos típicos: escaleno (3,4,5); equilátero (3,3,3); isósceles (3,3,4) y sus permutaciones; suma = tercero (1,2,3) y permutaciones; suma < tercero (12,15,30); isósceles degenerado (2a = c); lado cero, negativo o todos cero (error: lados > 0); valor no entero (3, 4.5, 5) y cantidad de valores incorrecta (3, 4) (formato inválido). **Lectura del archivo:** no existe, está vacío, separador distinto, letras, números enormes, líneas en blanco. Siempre con **salida esperada**.

Clases de equivalencia (válidas): 1 equilátero, 2 isósceles, 3 escaleno (todos > 0). **Inválidas:** 4 algún lado ≤ 0, 5 suma igual al tercero, 6 suma menor, 7 no entero, 8 cantidad de valores distinta de 3.

Defecto que los 14 casos no ven: `if ((a+b <= c) || ...)` con (2147483647, 2147483647, 1) desborda `int` (a + b da -2) y rechaza un isósceles válido. Corrección: `(long)a + b <= c` o comparar `a > c - b`.

8.10 Práctica: cotizador de envíos

Especificación: peso de 0,1 a 30,0 kg; servicio exacto Estándar / Express / Económico; código postal entero de 1000 a 9999. Base: liviano (hasta 5,0 kg) $3.000, pesado $6.000. Multiplicador: Económico × 0,8, Estándar × 1, Express × 1,5.

- **Clases válidas:** (1) 0,1 ≤ peso ≤ 5,0; (2) 5,1 ≤ peso ≤ 30,0 (se subdivide porque la tarifa base cambia); (6) Estándar, (7) Express, (8) Económico; (11) 1000 ≤ cp ≤ 9999.
- **Clases inválidas:** (3) peso < 0,1; (4) peso > 30,0; (5) no numérico; (9) otro texto; (10) vacío; (12) cp < 1000; (13) cp > 9999; (14) cp no numérico.
- **3 casos válidos alcanzan:** V1 (2,5; Estándar; 5000) → $3.000; V2 (12,0; Express; 1425) → $9.000; V3 (20,0; Económico; 2000) → $4.800.
- **8 inválidos, uno por clase:** peso 0,0 / 35,0 / abc; servicio Premium / vacío; cp 999 / 10000 / ABCD (cada uno con el resto válido; salida: error del dato inválido).
- **Siguiente paso, valores límite:** peso 0,0 · 0,1 · 5,0 · 5,1 · 30,0 · 30,1; cp 999 · 1000 · 9999 · 10000. Ambigüedad a detectar: ¿Express acepta mayúsculas/minúsculas?

Tips para el Parcial

- Frases trampa: «probar es demostrar que funciona» (falso). El éxito de una prueba = hallar un error.
- **Un caso inválido cubre UNA sola clase inválida**; los válidos se combinan. Y siempre escribí la **salida esperada**.
- Cobertura de condiciones **no implica** cobertura de decisiones (y viceversa); condición/decisión sí incluye ambas.
- Equivalencia vs. límite: representante vs. bordes (justo dentro y justo fuera).

9. NIVELES, TIPOS Y TÉCNICAS DE PRUEBA

9.1 Mapa general

Tres preguntas distintas: **nivel** (¿sobre qué alcance?), **tipo** (¿qué atributo se verifica?) y **técnica** (¿cómo se diseñan los casos?). Un mismo caso se describe con las tres miradas.

Niveles	Tipos	Técnicas
Unidad,	Funcional, humo y regresión,	Partición de equivalencia, valores

Niveles	Tipos	Técnicas
Integración, Sistema, Aceptación	performance y esfuerzo, seguridad y usabilidad	límite, conjetura de errores, cobertura lógica

Modelo en V: cada nivel valida una etapa: Unidad ↔ codificación; Integración ↔ diseño de módulos; Sistema ↔ objetivos del sistema; Aceptación (UAT) ↔ requisitos del usuario.

9.2 Pruebas de unidad (unit testing)

- Probar clases, funciones o subrutinas **aisladas** del resto. Foco en poco código, facilita el debugging, permite paralelizar. **Caja blanca**, pero los datos pueden salir de caja negra. Las define el programador.
- **Buena prueba de unidad:** automatizable (CI/CD), independiente (mocks/stubs, sin BD) y que documenta el comportamiento. **FIRST: F**ast, **I**ndependent, **R**epeatable, **S**elf-validating, **T**imely.
- **Estructura AAA:** Arrange (preparar) → Act (ejecutar) → Assert (verificar). Nombre: Método_Escenario_Resultado. Una prueba parametrizada ([Theory]) corre varios casos.
- **Stub vs. mock:** el **stub** devuelve datos preparados (verificación de **estado**, simple y pasivo); el **mock** además **verifica cómo fue usado** (comportamiento; falla si no se llama al método esperado).
- **Pirámide de pruebas:** muchas de unidad (rápidas, baratas), algunas de integración, pocas extremo a extremo (lentas, costosas, frágiles). Una pirámide invertida hace lento y caro cada cambio.

9.3 Pruebas de integración

Verifican que módulos **ya probados por separado** funcionan juntos. Errores típicos: contratos de datos (formato, tipos, nombres), orden y estado de las llamadas, manejo de errores (timeouts, reintentos), configuración (permisos, versiones, zonas horarias).

	No incremental (big-bang)	Incremental
Cómo	Cada módulo se prueba **solo** (con drivers y stubs) y **después se combinan todos** a la vez	Se agrega el siguiente módulo al conjunto ya probado y recién entonces se prueba
Ventaja	Permite probar módulos en paralelo	La falla apunta al **último módulo agregado**; errores de interfaz aparecen antes; los primeros módulos se prueban más veces
Problema	Si falla al combinar, ¿cuál es el culpable?	Puede ser más lento de armar

En general se prefiere la **incremental**. Estrategia **híbrida:** combina ambas, probando individualmente los componentes más importantes.

- **Descendente (top-down):** se empieza por el módulo superior y se baja, usando **stubs** (simulan al módulo **llamado**). + Prueba temprano la lógica de control; − los stubs pueden ser complejos.

- **Ascendente (bottom-up):** se empieza por las hojas y se sube, usando **drivers** (simulan al **llamador**). + No requiere stubs y prueba a fondo los módulos base; − el programa completo recién existe al final.

9.4 Función, humo, sanidad y regresión

- **Prueba de función:** busca discrepancias entre el sistema y su **especificación externa** (requisitos funcionales). Entradas válidas e inválidas; **caja negra** (equivalencia, límites, conjetura). Ejemplo: carrito (agregar, quitar, stock 5 con 6 unidades → rechaza; con 5 → acepta).
- **¿Cómo documentar un caso?** Nombre definido, trazabilidad con el CU, historial de cambios, objetivo claro, precondiciones, **resultado esperado por paso**, registro del resultado, de quien lo ejecutó y de los ciclos ejecutados.

Tipo	Pregunta	Alcance	Cuándo
Humo	¿Arranca y funciona lo básico?	Amplio y superficial	En cada versión o despliegue
Sanidad	¿El cambio puntual anda?	Acotado a un área	Después de un arreglo
Regresión	¿Se rompió algo que andaba?	Amplio y profundo	Después de cada cambio

Si falla la prueba de humo, no tiene sentido seguir probando. Las tres son candidatas a automatizarse en CI. Humo, ejemplo: abrir la app, iniciar sesión, buscar un producto, agregar al carrito.

9.5 Pruebas de sistema

- **Objetivo:** comparar el sistema con sus **objetivos originales**. Es la más difícil y la peor interpretada: **no** es repetir pruebas funcionales o de integración. Se prueba el sistema completo, en un entorno parecido a producción, incluyendo requisitos funcionales y **no funcionales**. Los casos se diseñan a partir de los **objetivos** (no de la especificación externa); la documentación para el usuario sirve de contraste.
- **Función vs. sistema**: función mira cada funcionalidad aislada (existe y hace lo especificado); sistema mira el conjunto desde lo deseable (por ejemplo, la factura se imprime desde donde y cuando le resulta cómodo al usuario).

Tipo	Qué se aumenta / qué busca	Ejemplo
Alcance	¿Está implementado todo lo prometido? Se puede hacer sin computadora	Exportar a PDF: falta
Volumen	Cantidad de **datos**	10 millones de registros; reporte de 10 años
Esfuerzo	**Carga en poco tiempo** (pico o más allá del límite)	5.000 usuarios en 1 minuto (JMeter)

Tipo	Qué se aumenta / qué busca	Ejemplo
Performance	Carga prevista, ¿cumple **tiempos objetivo**? Mide tiempo de respuesta, throughput, recursos, estabilidad	500 usuarios, respuesta < 2 s
Facilidad de uso	Tiempo de aprendizaje y de ejecución, tasa de errores, satisfacción (SUS), retención	Usuarios nuevos completan tareas básicas
Seguridad	Vulnerabilidades, controles (autenticación, autorización, cifrado, auditoría), cumplimiento (OWASP, PCI DSS, Ley 25.326)	Intentar burlar los controles
Otros	Configuración, compatibilidad y conversión, instalación, recuperación (recovery), documentación	Manuales que coinciden con el sistema

Pentesting: simula un ataque real. Fases: reconocimiento, escaneo, explotación, post-explotación, informe. Caja negra (sin información), gris (parcial) o blanca (código y arquitectura). Requiere **autorización escrita** y alcance acordado. Las pruebas de almacenamiento verifican **límites de memoria/almacenamiento**.

9.6 Pruebas de aceptación (UAT - User Acceptance Testing) y de instalación

- Las realiza el **usuario final o cliente** para verificar y aceptar el sistema antes de producción, en un entorno independiente con datos similares a producción; es la fase **final**, después de función, integración y sistema. Variantes: contractual, **alfa** (dentro de la organización; el usuario está presente) y **beta** (usuarios reales en su entorno, sin supervisión directa).
- **Criterios de aceptación:** se escriben **antes** de construir, con el cliente, en lenguaje de negocio: **Dado** (contexto) **Cuando** (acción) **Entonces** (resultado observable).
- **Pruebas de instalación:** buscan errores de instalación, actualización y desinstalación (rutas, permisos, dependencias, mensajes de error, reinicio, **rollback**). No se asocian a una fase específica; sentido principal para apps desktop.

9.7 ¿Cuándo terminar las pruebas?

- **Criterios de poco sentido:** (a) terminar cuando se acaba el tiempo; (b) cuando se ejecutan todos los casos sin detectar errores (incentiva casos débiles); (c) cuando los casos de límites y conjetura no encuentran nada.
- **Criterios en términos positivos:** (1) estimar cuántos errores hay y terminar al encontrar cierta proporción; (2) la **curva de errores** por unidad de tiempo baja y se aplana; (3) **objetivos de calidad**: cobertura alcanzada y cero defectos críticos abiertos.
- **Razones de clase:** *políticas* (deadline impuesto, fecha de elecciones), *técnicas* (registro diario de errores; **siembra de errores:** insertar errores a propósito y estimar los reales según qué proporción de los sembrados se encuentra) y *económicas* (costo-beneficio y rentabilidad del proyecto). El mejor criterio suele ser una **combinación**; el defecto del criterio matemático es que es poco predictivo para un contrato con fecha.
- **Plan de pruebas (mínimo):** objetivos, criterios de terminación, cronograma, responsabilidades y asignación de recursos.

9.8 Secuencia, tipos de errores y alfa/beta

- **Secuencia:** prueba de escritorio (programador) → inspección → unidad → incremental / integración → **(nivel inferior: equipo de desarrollo)** → función → sistema → facilidad de uso → instalación, etc. → alfa → beta **(nivel superior, con usuario)**. Hay que equilibrar incrementales y de unidad.
- **Errores de nivel inferior** (presupuestados, medibles, los ven solo profesionales): *triviales* (sintaxis, baratos) y *de lógica* (el sistema no hace lo que se quiere).
- **Errores de nivel superior** (difíciles de medir, los percibe el cliente y generan pérdida económica, de prestigio y confianza): *de sistema* (hay que rehacer gran parte) y *de comunicación* (hace lo que el cliente quiere, pero no como él quiere).
- **Alfa/beta:** si los hace la organización, los errores cuentan como de nivel inferior; si los hace el público (beta pública), pasan a ser de nivel superior.
- **¿Dónde probar más?** En componentes con más errores previos y donde trabajó mucha gente. Causas de errores: estado anímico del programador, desconocimiento del tema, desconocimiento del entorno.

9.9 Depuración (debugging)

- **Prueba** detecta errores (la hace otro); **depuración** encuentra y corrige (la hace el programador, que es quien mejor sabe dónde buscar). Pasos: (1) determinar naturaleza y ubicación exacta, (2) corregir.
- **Métodos no sistemáticos (fuerza bruta):** volcado de memoria (imagen estática), **esparcimiento de sentencias** (prints; cambia el código y puede enmascarar el error) y herramientas automáticas (puntos de detención). Solo como suplemento o si fallan los demás.
- **Por inducción** (de lo particular a lo general): partir de los síntomas; preguntas: qué, cuándo, dónde, con qué alcance; formular y probar hipótesis.
- **Por deducción** (de lo general a lo particular): enumerar causas posibles, eliminarlas con datos, refinar la restante y probarla.
- **Rastreo hacia atrás** (programas pequeños) y **por medio de pruebas** (casos variantes para ubicar el error).
- **Principios de búsqueda:** pensar; si hay impasse, descansar o explicar el problema a otro; herramientas solo como segundo recurso; evitar experimentos.
- **Principios de reparación:** donde hay una falla probablemente hay otra; arreglar el **error**, no un síntoma; la corrección **no es 100% segura** y su probabilidad baja en programas grandes; puede producir **nuevos errores**; la reparación debe volver temporalmente al diseño. Es decir, un arreglo mal hecho puede empeorar el sistema, no que «siempre» lo empeore.

Tips para el Parcial

- Top-down = stubs; bottom-up = drivers. Incremental: el sospechoso es el último módulo agregado.
- Humo (amplio/superficial) ≠ sanidad (acotado) ≠ regresión (amplio/profundo).
- Volumen = datos; esfuerzo = pico; performance = tiempo objetivo.
- Según el profesor, **no se toma un listado de pruebas de nivel superior**.
- Prueba ≠ depuración: detectar vs. encontrar y corregir. Siembra de errores ≠ esparcimiento de sentencias.

10. IMPLEMENTACIÓN Y DESPLIEGUE: DE LAS PRUEBAS A PRODUCCIÓN

10.1 Qué es y sus cinco etapas

- **Implementación:** poner en funcionamiento un sistema **ya probado**, reemplazando al anterior **sin interrumpir la operación**. En clase: llevar al sistema a una condición de **régimen** (procesando sin errores el volumen para el que fue diseñado). ***Despliegue*** = insertar el producto en el mundo real; según la bibliografía ambos términos pueden usarse como similares.
- **Etapas** (pueden ser secuenciales o simultáneas): (1) **conversión de programas**, (2) **conversión de datos**, (3) **conversión de procedimientos**, (4) **capacitación a usuarios**, (5) **evaluación final**. Ocurre en paralelo a la **gestión de liberaciones** (release).

10.2 Conversión de programas: 5 estrategias

Estrategia	Qué es	Cuándo conviene	Ventajas	Riesgos y costos
Total (big bang)	Un sistema reemplaza al otro en una **fecha de corte**	Sistemas triviales, el anterior colapsó, lo impone la ley	Simple, barato, sin dos sistemas que sincronizar	Todo el riesgo en un día; volver atrás es difícil; exige ensayo y plan de contingencia
En paralelo	Viejo y nuevo conviven y **se comparan**	Sistemas críticos donde un error es inaceptable (sueldos, contabilidad, bancos)	Red de seguridad, permite comparar, baja el riesgo operativo	**Cara:** doble operación y doble carga; requiere gran manejo del personal
Gradual	Componentes nuevos sustituyen a los viejos **de a uno**	Sistemas modulares donde el riesgo se reparte en el tiempo	Riesgo repartido, se aprende de cada etapa	Problemas de acople entre módulos viejos y nuevos; funciones de sincronización que luego se descartan
Por prototipado	Las funcionalidades surgen **a demanda** del usuario con prototipos evolutivos	Requerimientos poco claros o cambiantes, usuarios no técnicos	Detecta problemas de requerimientos desde el inicio	Alcance difuso si no se controla

Estrategia	Qué es	Cuándo conviene	Ventajas	Riesgos y costos
Distribuida	Se implanta **sede por sede**, empezando por las más pequeñas	Instituciones con sucursales (bancos, cadenas, universidades)	Errores donde el impacto es menor; se escala con la experiencia	Conviven versiones y procesos distintos; ciclo largo; soporte en cada etapa

Estrategia	Riesgo	Costo	Duración	Vuelta atrás
Total	Alto	Bajo	Corta	Difícil
En paralelo	Bajo	Alto	Media	Inmediata
Gradual	Medio	Medio	Larga	Por módulo
Prototipado	Bajo en requisitos	Medio	Iterativa	No aplica
Distribuida	Medio	Medio	Larga	Por sede

¿Cómo elegir? (1) ¿un error es inaceptable (dinero, vidas, legal)? → **paralelo**; (2) ¿se divide en módulos? → **gradual**; (3) ¿requerimientos inciertos? → **prototipado**; (4) ¿muchas sedes de distinto tamaño? → **distribuida**; (5) ¿fecha impuesta o sistema simple? → **total**. Pesan también presupuesto, capacidad del personal y claridad de requerimientos. Las estrategias se pueden **combinar** (ej.: distribuida + paralela en cada sede).

Ejercicio resuelto: 1) liquidación de sueldos → **paralelo**; 2) 120 farmacias → **distribuida**; 3) facturación electrónica con fecha límite → **total**; 4) portal municipal con requerimientos difusos → **prototipado**; 5) ERP por módulos → **gradual**.

10.3 Conversión de datos (migración)

- Los datos sobreviven a los sistemas. Proceso: **Extraer** → **Transformar** (limpiar, mapear, convertir) → **Cargar** → **Validar** (reconciliar y aprobar).
- **Seis pasos para no perder información:** inventariar fuentes; limpiar y depurar (duplicados, nulos, formatos); mapear campos y reglas; **ensayar** la migración con datos reales; **reconciliar** (cantidad de registros y totales de control: origen = destino); plan de corte y **vuelta atrás**.
- Requiere conocer las tecnologías de origen y destino. El costo puede **igualar al de desarrollar el sistema**: se planifica, se estima y se presupuesta desde el inicio. Con datos en papel, prever digitalización y personal dedicado. Dejar claro con el cliente **quién migra** los datos; puede haber información que existe solo «en la cabeza de alguien» o datos nuevos que antes no se registraban.

10.4 Conversión de procedimientos

Los procedimientos de trabajo cambian o se rediseñan (**reingeniería de procesos**; ej.: pedido en papel → pedido online con aprobación digital). Puede ser traumático (rutinas, roles,

puestos). Los empleados deben **adoptarlos** o el sistema no rinde. **Gestión del cambio:** explicar el porqué, involucrar a usuarios clave, acompañar y medir la adopción.

10.5 Capacitación a usuarios

- **Operativa** (quienes usan el sistema día a día): qué hace cada función y **cómo** ejecutar cada tarea; práctica con casos reales en un entorno de prueba; manuales, guías, tutoriales.
- **Ejecutiva** (quienes deciden y patrocinan): **qué** problemas resuelve, beneficios, indicadores y reportes, qué cambia en la organización; es «vender el producto» (acción de marketing).
- **Cómo y cuándo:** presencial u online; **sandbox** con datos de ejemplo; **capacitar capacitadores**; formal **antes** del corte, soporte en sitio **durante**, refuerzo y consultas **después**.
- **Mesa de ayuda / centro de información**: canaliza las dudas simples de los usuarios para no distraer al equipo de implementación. Requiere **solvencia técnica** para filtrar y resolver consultas.

10.6 Evaluación final

Cierra la implementación: ¿el sistema hace lo prometido, al costo y en el plazo previstos? Se revisan: requisitos y aceptación, adopción y satisfacción de usuarios, objetivos de negocio, costo y plazo, integridad de la migración y **lecciones aprendidas**.

10.7 Gestión de liberaciones (release management)

- **Ciclo de cada versión:** Iniciación → Planificación y diseño → Desarrollo → Testing → Despliegue → Post-deploy (y lo aprendido vuelve a la iniciación: el ciclo es continuo).
- **Iniciación:** llegan solicitudes (nuevas características, bugs, cambios) que se **evalúan** por impacto en el negocio, prioridad, factibilidad y costo, y se **aprueban, postergan o rechazan**.
- **Plan del release:** identificación (nombre, fecha), cronograma (QA, aprobación final, fecha de despliegue), alcance y responsables (objetivos, límites, roles) y operación (entornos, estrategia de despliegue, **plan de contingencia**, monitoreo y soporte).
- **Construcción y puesta:** diseñar y construir (pruebas de unidad e integración) → **testing** (sistema, regresión, UAT, aprobación final) → **deployment** (ejecutar el plan, capacitar, comunicar la nota de release). Una versión no está terminada cuando el código funciona, sino cuando los usuarios **pueden usarla**.

Estrategia de despliegue	Cómo funciona	Pros / contras
Rolling	Se actualizan pocos servidores por vez	Sin cortes, pero conviven dos versiones
Canary	La nueva versión llega primero a un pequeño porcentaje de usuarios	Detecta problemas con poco impacto
Blue-green	Dos entornos idénticos; se cambia el tráfico del azul al verde	Vuelta atrás inmediata, pero duplica infraestructura

Estrategia de despliegue	Cómo funciona	Pros / contras
Feature flags	El código se despliega apagado y se enciende por funcionalidad	Separa **desplegar** de **liberar**

Toda estrategia necesita un **rollback** definido y probado **antes** de desplegar.

- **Post-deployment: monitorear** (errores, rendimiento, disponibilidad, tickets, hotfix ante fallas críticas, soporte reforzado) y **documentar** (lecciones aprendidas, desvíos de plan/costo/calidad, mejoras).
- **Tipos de release por alcance: MAJOR** (v1.0 → v2.0: cambios grandes, posibles incompatibilidades), **MINOR** (v2.1 → v2.2: funcionalidades nuevas compatibles), **PATCH** (v2.2.1 → v2.2.2: corrección de errores sin cambiar funcionalidad).
- **Por madurez: Alpha** (interna, incompleta, inestable) → **Beta** (funcionalidad completa con errores conocidos, usuarios seleccionados) → **Release Candidate** (solo se corrigen bugs críticos) → **GA / General Availability** (estable, para todos). Ej.: 3.0.0-alpha.1 → 3.0.0-beta.2 → 3.0.0-rc.1 → 3.0.0.
- **Nota de release (7 partes):** información general (nombre, versión, fecha, tipo, audiencia), resumen ejecutivo, detalle de cambios (nuevo, mejora, corrección), impacto para el usuario, instrucciones de actualización, problemas conocidos y contacto. Buena práctica: escribirla para la audiencia, en lenguaje claro y con versiones consistentes.

Tips para el Parcial

- Justificá la estrategia con **ventaja buscada + riesgo aceptado**. Paralelo = seguridad a costo alto; total = simple pero todo el riesgo en un día.
- Datos, procedimientos y personas son **tan importantes como el programa**.
- Blue-green vuelve atrás al instante; canary limita el impacto; rolling no tiene cortes; feature flags separan desplegar de liberar.
- Major/minor/patch hablan de **alcance**; alpha/beta/RC/GA hablan de **madurez**.
- El **mantenimiento no entra** en el parcial (según lo anotado en clase).